# Overriding vs Overloading Methods

## Lecture 18

Based on Slides of Dr. Norazah Yusof

# The `Object` Class

Every class in Java is descended from the java.lang.Object class. If no inheritance is specified when a class is defined, the superclass of the class is **Object**.

```
public class Circle {
   ...
}
```

Equivalent

```
public class Circle extends Object {
   ...
}
```

# Declaring a Subclass

**A subclass extends properties and methods from the superclass. You can also:**

- **Add new properties**
- **Add new methods**
- **Override the methods of the superclass**

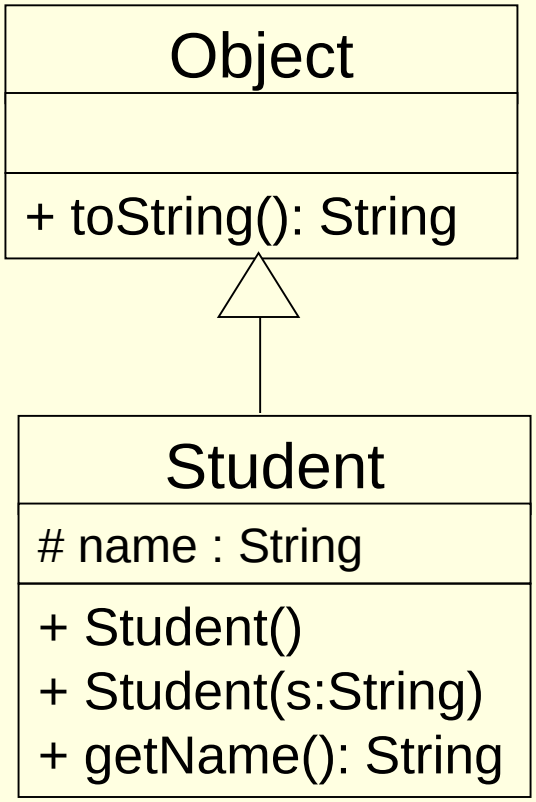
# The toString() method in Object

**The toString() method returns a string representation of the object. The default implementation returns a string consisting of a class name of which the object is an instance, the at sign (@), and a number representing this object.**

```
Circle circ = new Circle();
System.out.println(circ.toString());
```

**The code displays something like Circle@82ba41 . This message is not very helpful or informative. Usually you should override the toString method so that it returns a digestible string representation of the object.**

# Example 1:




**By default, all Java classes are subclasses of the `Object` class (the most general class in Java's hierarchy).**

**One public method that is defined in the `Object` class is the `toString()` method.**

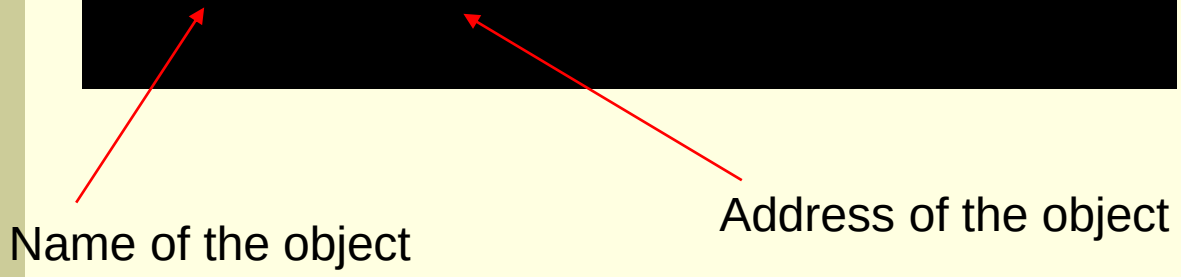
# Example 1:

```
public class Student
{   protected String name;
    public Student () {   }
    public Student (String s) {
        name = s;
    }
    public String getName() {
        return name;
    } }
```

```
public class TestStudent1 {
    public static void main( String args[]){
     Student stu = new Student("Ana");
     System.out.println (stu.toString());
}
```

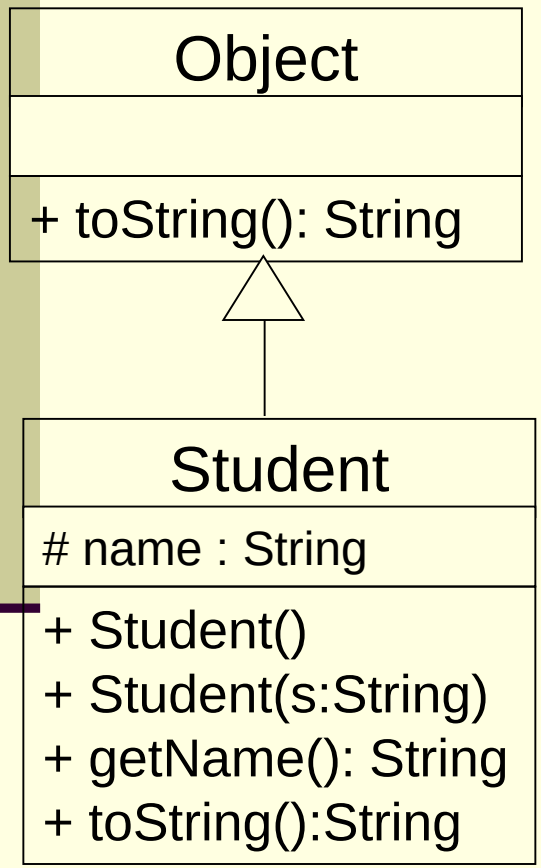
# Example 1: output

```
Student@82ba41
```




**The default implementation of `toString()`**
**Returns the name of the object's class and the address where the object is stored in the memory.**

# Example 2: Overriding an Inherited Method




**`toString()` method is redefined in subclasses of `Object` class.**

**Overriding `toString()` in a Subclass provides a customized string representation of the `Object` in that subclass.**

# Example 2

```
public class Student
{  protected String name;
    public Student () {   }
    public Student (String s) {
        name = s;
    }
    public String getName() {
        return name;
    }
    public String toString() {
        return "My name is " + name + " and I am a
   Student.";
    }
}
```

# Example 2

```
public class TestStudent2 {
   public static void main( String args[]){
    Student stu = new Student("Ana");
    System.out.println (stu.toString());
   }
}
```

Output:

```
My name is Ana and I am a Student.
```

# Overriding Methods in the Superclass

- Any time you have a class that inherits a method from a superclass, you have the opportunity to override the method (unless, the method is marked final).
- The key benefit of overriding is the ability to define behavior that's specific to a particular subclass type.

```
public class Animal {
   public void eat() {
      System.out.println("Generic Animal Eating Generically");
   }
}
class Horse extends Animal {
   public void eat() {
       System.out.println("Horse eating hay, oats, "
                          + "and horse treats");
   }
   public void buck() {
      System.out.println ("This is how I jump");
   }
}
```

# Overriding Methods in the Superclass

```
public class TestAnimals {
  public static void main (String [] args) {
    Animal a = new Animal();
    Animal b = new Horse();  //Animal ref, but a Horse object
    a.eat(); // Runs the Animal version of eat()
    b.eat(); // Runs the Horse version of eat()
  }
}
```

- The Animal class creator might have decided that for the purposes of polymorphism, all Animal subtypes should have an eat() method defined in a unique, specific way.
- Polymorphically, when someone has an Animal reference that refers not to an Animal instance, but to an Animal subclass instance, the caller should be able to invoke eat() on the Animal reference, but the actual runtime object (say, a Horse instance) will run its own specific eat() method.

# Overriding Methods in the Superclass

```
public class TestAnimal2 {
  public static void main (String [] args) {
    Animal c = new Horse();  //Animal ref, but a Horse object
    c.buck();
  }
}
```

- The TestAnimal2 class uses an Animal reference to invoke a method on a Horse object.
- Remember, the compiler will allow only methods in class Animal to be invoked when using a reference to an Animal.
- Can't invoke buck(); because **Animal class doesn't have that method**.

# Dynamic Binding

- **A method call is bound to the correct implementation of the method at runtime by the Java Virtual Machine (JVM).**
- **When JVM encounters a method call, it uses information about the class hierarchy to bind the method call to the correct implementation of that method.**
- **Java's dynamic-binding mechanism, which is also called late binding, or runtime binding, leads to what is known as polymorphism.**

# NOTE

- **An instance method can be overridden only if it is accessible. Thus a private method cannot be overridden, because it is not accessible outside its own class.**

# A private method cannot be overridden

```
public class TestAnimals3 {
  public static void main (String [] args) {
    Horse h = new Horse();
    h.eat();
  }
}
class Animal {
  private void eat() {
    System.out.println("Generic Animal Eating Generically");
  }
}
class Horse extends Animal { }
}
```

Method eat() in the superclass can't be inherited, because it is private. Therefore, method eat() cannot be overridden.

# NOTE

- **If a method defined in a subclass is private in its superclass, the two methods are completely unrelated.**

# A private method cannot be overridden

```
public class TestAnimals2 {
  public static void main (String [] args) {
    Animal a = new Animal();
    Animal b = new Horse();  //Animal ref, but a Horse object
    a.eat(); // Runs the Animal version of eat()
    b.eat(); // Runs the Horse version of eat()
  }
}
class Animal {
  public void eat() {
    System.out.println("Generic Animal Eating Generically");
  }
}
class Horse extends Animal {
  private void eat() {  // whoa! - it's private!
    System.out.println("Horse eating hay, oats, "
                       + "and horse treats");
  }
}
```

# Overloading Method

# Overriding Versus Overloading

- **Do not confuse *overriding* a method in a derived class with *overloading* a method name**
  - **When a method is overridden, the new method definition given in the derived class has the exact same number and types of parameters as in the base class**
  - **When a method in a derived class has a different signature from the method in the base class, that is overloading**
  - **Note that when the derived class overloads the original method, it still inherits the original method from the base class as well**

# Overriding vs. Overloading

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
  }
}

class B {
  public void p(int i) {
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(int i) {
    System.out.println(i);
  }
}
```

**The method p(int i) in class A overrides the same method defines in class B.**

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
  }
}

class B {
  public void p(int i) {
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

**The method p(int i) in class A overloads th same method defines in class B.**

# Overloading

- **Over*loaded* methods let you reuse the same method name in a class, but with different arguments (and optionally, a different return type).**

# Overloading rules

- **Overloaded methods MUST change the argument list.**
- **Overloaded methods CAN change the return type.**
- **Overloaded methods CAN change the access modifier.**
- **Overloaded methods CAN declare new or broader checked exceptions.**
- **A method can be overloaded in the *same* class or in a *subclass*.**

# Legal Overloads

```
public void changeSize(int size, String
  name, float pattern) { }
```

- **The following methods are legal overloads of the changeSize() method:**

```
public void changeSize(int size, String
  name) { }
public int changeSize(int size, float
  pattern) { }
public void changeSize(float pattern,
  String name){ }
```

# Class Adder

```
public class Adder {
  public int addThem(int x, int y) {
    return x + y;
  }

  // Overload the addThem method
  public double addThem(double x, double y)
  {
    return x + y;
  }
}
```

# Class TestAdder

```
public class TestAdder {
  public static void main (String [] args) {
    Adder a = new Adder();
    int b = 27;
    int c = 3;

 // Which addThem is invoked?
    int result = a.addThem(b,c);
    double doubleResult = a.addThem(22.5,9.3);

    System.out.println (result);
    System.out.println (doubleResult);
   }
}
```

# Invoking overloaded methods that take object references

```
class Animal { }
class Horse extends Animal { }

public class UseAnimals {
    public void doStuff(Animal a) {
        System.out.println("In the Animal
  version");
    }
    public void doStuff(Horse h) {
        System.out.println("In the Horse
  version");
    }
}
```

# Invoking overloaded methods that take object references

```
public class TestUseAnimals2 {
    public static void main (String [] args)
  {
      UseAnimals ua = new UseAnimals();
      Animal animalobj = new Animal();
      Horse horseobj = new Horse();
      Animal animalRefToHorse = new Horse();

      ua.doStuff(animalobj);
      ua.doStuff(horseobj);
      ua.doStuff(animalRefToHorse);
    }
}
```

**Table 2-3: Differences Between Overloaded and Overridden Methods**

| | Overloaded Method | Overridden Method |
|---|---|---|
| Argument (s) | Must change. | Must not change. |
| Return type | Can change. | Can't change except for covariant returns. |
| Exceptions | Can change. | Can reduce or climinate. Must not throw new or broader checked exceptions. |
| Access | Can change. | Must not make more restrictive (can be less restrictive). |
| Invocation | *Reference* type determines which overloaded version (based on declared argument types) is selected. Happens at *compile* time. The actual *method* that's invoked is still a virtual method invocation that happens at runtime, but the compiler will already know the *signature of* the method to be invoked. So at runtime, the argument match will already have been nailed down, just not the *class* in which the method lives. | *Object* type (in other words, *the type of the actual instance on the heap)* determines which method is selected. Happens at *runtime*. |

# Exercises (Lab 8)

- **Do Exercise 2:**
  - **Question 11, page 164**
  - **Question 13, page 165**
  - **Question 14, page 166**
- **Take home:**
  - **Question 3, page 153**